Intern Name: Noor-ul-huda

Project Name*:* Supervised Learning (Fraud Detection Pipeline)

Dataset Used*:* Credit Card Fraud Detection (284,807 rows).

How I Built It & Results:

The biggest challenge with this dataset is that fraud is extremely rare, making up less than one percent of the data. If you train a model on it in the usual way, it cheats by guessing "not fraud" every time. This makes it seem highly accurate while being completely ineffective. To address this, I first set aside twenty percent of the dataset. This gave the models a completely unbiased final test. Next, I used SMOTE on the training data to create realistic synthetic fraud cases until the split was an even fifty-fifty. This approach made the algorithms learn the actual patterns of fraud instead of ignoring them. I tested two different methods. The Logistic Regression model was fast and reliable, achieving an ROC-AUC score of 0.9700. However, the Random Forest Classifier outperformed it, scoring 0.9777 and catching eighty-five percent of the hidden fraud cases.

```python
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, roc_auc_score
from imblearn.over_sampling import SMOTE
df = pd.read_csv('creditcard.csv')
print("--- Dataset Breakdown ---")
print(f"Total Rows: {df.shape[0]}, Total Columns: {df.shape[1]}")
print(df['Class'].value_counts())
scaler = StandardScaler()
df['Amount'] = scaler.fit_transform(df['Amount'].values.reshape(-1, 1))
df = df.drop(['Time'], axis=1)
X = df.drop(['Class'], axis=1)
y = df['Class']
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y
)
smote = SMOTE(random_state=42)
X_train_balanced, y_train_balanced = smote.fit_resample(X_train, y_train)
print("\n--- Balanced Training Data Counts ---")
print(pd.Series(y_train_balanced).value_counts())
lr_model = LogisticRegression(max_iter=1000, random_state=42)
lr_model.fit(X_train_balanced, y_train_balanced)
rf_model = RandomForestClassifier(n_estimators=100, max_depth=10, random_state=42, n_jobs=-1)
rf_model.fit(X_train_balanced, y_train_balanced)
lr_preds = lr_model.predict(X_test)
lr_probs = lr_model.predict_proba(X_test)[:, 1]
print("\n" + "="*40)
print("LOGISTIC REGRESSION PERFORMANCE REPORT")
print("="*40)
print(classification_report(y_test, lr_preds))
print(f"ROC-AUC Score: {roc_auc_score(y_test, lr_probs):.4f}")
rf_preds = rf_model.predict(X_test)
rf_probs = rf_model.predict_proba(X_test)[:, 1]
print("\n" + "="*40)
print("RANDOM FOREST PERFORMANCE REPORT")
print("="*40)
print(classification_report(y_test, rf_preds))
print(f"ROC-AUC Score: {roc_auc_score(y_test, rf_probs):.4f}")
```

```
--- Dataset Breakdown ---
Total Rows: 284807, Total Columns: 31
Class
0    284315
1       492
Name: count, dtype: int64

--- Balanced Training Data Counts ---
Class
0    227451
1    227451
Name: count, dtype: int64

========================================
LOGISTIC REGRESSION PERFORMANCE REPORT
========================================
              precision    recall  f1-score   support

           0       1.00      0.97      0.99     56864
           1       0.06      0.92      0.11        98

    accuracy                           0.97     56962
   macro avg       0.53      0.95      0.55     56962
weighted avg       1.00      0.97      0.98     56962

ROC-AUC Score: 0.9700

========================================
RANDOM FOREST PERFORMANCE REPORT
========================================
              precision    recall  f1-score   support

           0       1.00      1.00      1.00     56864
           1       0.43      0.86      0.57        98

    accuracy                           1.00     56962
   macro avg       0.71      0.93      0.79     56962
weighted avg       1.00      1.00      1.00     56962

ROC-AUC Score: 0.9777
```